	**Fundamentos de Sistemas Embebidos**
Facultad de Ingeniería	UNAM

Laboratorio de Fundamentos de Sistemas Embebidos

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

No de Práctica(s): 7

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

No. de brigada: 12

Semestre: 9°

Fecha de entrega: 27 de octubre del 2026

Observaciones:

CALIFICACIÓN: _____

Reconocimiento de Voz y de la Mano con IA

Objetivos

Resultados y Análisis

Configuración del sistema

Para preparar el entorno de la práctica sin depender del router del laboratorio, se configuró el acceso Wi-Fi directamente en la microSD de la Orange Pi 2W. Esto implicó crear un archivo `.nmconnection` en `/etc/NetworkManager/system-connections/` asignando un UUID único, el SSID, la contraseña de la red y un nivel de `autoconnect-priority` alto (ej. 10) para priorizarla sobre otras redes. Se verificó que los permisos del archivo fueran 600 con propietario `root:root` para que el demonio NetworkManager lo detectara correctamente durante el arranque.

Adicionalmente, se revisó el proceso de instalación del intérprete de comandos Zsh mediante el gestor de paquetes, incluyendo la descarga del framework Oh My Zsh y la clonación del plugin `zsh-autosuggestions` para habilitar el autocompletado predictivo. Aunque la documentación indica cómo cambiar el shell predeterminado con el comando `chsh`, se optó por conservar Bash por defecto. Esta decisión técnica priorizó la consistencia con el entorno de Fedora Linux configurado en el equipo principal, conservando intacto el prompt personalizado con formato ANSI de 256 colores que ya opera en el archivo `.bashrc`.

Caso A - python 2_puntos.py

A.1 Las herramientas de la terminal	
1. `python 2_puntos.py`. ¿Devuelve el prompt? Intenta escribir algo	Si deja escribir, pero no responde al input
2. `Ctrl+Z`. ¿Qué pasa con los puntos?	Lo suspende, los puntos se detienen
3. `jobs`. ¿Qué dice del trabajo?	[1]+ Stopped python 2_puntos.py
4. `bg`. ¿Y ahora los puntos?	Los puntos reanudan su ejecución en segundo plano
5. `jobs`. ¿Cambió lo que dice?	Si, [1]+ Running python 2_puntos.py &
6. `fg`. ¿Qué pasa con el prompt?	Se mueve al primer plano, el prompt vuelve a estar activo
7. `Ctrl+C`. ¿Qué pasa con los puntos?	Lo interrumpe, los puntos se detienen

A.2 Las herramientas del sistema	
8. Lánzalo otra vez y recupera el prompt con `Ctrl+Z` y `bg`	Al ejecutar de nuevo, se detuvo el programa con `Ctrl+Z` y se reanudó con `bg`
9. `ps aux | grep .py` (1). Anota el PID (segunda columna):	3071
10. `kill -STOP <PID>`. ¿Los puntos? ¿Y qué dice `jobs`?	Los puntos se detienen y `jobs` muestra que están detenidos [1]+ Stopped python 2_puntos.py
11. `kill -CONT <PID>`. ¿Los puntos?	Los puntos se reanudan
12. `kill <PID>`. ¿Los puntos?	Se vuelven a detener
13. `ps aux | grep .py` (1). ¿Qué devuelve ahora?	[1]+ Terminated python 2_puntos.py muestra que se ha terminado
A.3 ¿Sobrevive a la terminal?	
14. Lánzalo otra vez. Cierra la ventana de la terminal y vuelve a entrar	Se ejecuto, se cerro la terminal con `CTRL+D` y se volvió a entrar
15. `ps aux | grep .py` (1). ¿Sobrevivió?	El proceso no sobrevivió al cierre de la terminal

¿Qué herramientas funcionaron en este caso?

Las herramientas que funcionaron en este caso fueron `ps`, `kill`, `jobs`, `fg` y `bg`. Las cuales son comandos básicos para administrar procesos en la terminal. De igual forma este caso nos ayudo a comprobar atajos de teclado para dormir o matar procesos.

Caso B - python 2_puntos.py &

1. `python 2_puntos.py &`. ¿Devuelve el prompt? Intenta escribir algo	Al añadir `&` al final del comando, el prompt se devuelve inmediatamente. Pasando el proceso a segundo plano.
2. `jobs`. ¿Qué dice? Compáralo con lo que decía en el paso 5 del caso A	El proceso se muestra como corriendo en segundo plano. Exactamente como en el caso A.
3. `fg`. ¿Puedes escribir ahora? ¿En qué caso acabas de convertirlo?	Al ingresar `fg`, el proceso se mueve a primer plano y puedes escribir. Similar al primer inciso del caso A.

4. `Ctrl+Z` y luego `bg`. ¿Recuperaste el prompt? ¿En qué caso estás ahora?	Al presionar `Ctrl+Z`, el proceso se detiene y al ejecutar `bg`, se reanuda en segundo plano. Similar al insico 5 del caso A.
5. `jobs`. ¿Se distingue de como estaba en el paso 2?	No se distingue ya que es la misma salida, indica que esta corriendo y en segundo plano.
6. `kill %1`. ¿Funciona? ¿Y los puntos?	El comando es funcional y los puntos se detienen.

En qué se diferencia de verdad del caso A?

El caso B se diferencia del caso A en que el proceso se ejecuta en segundo plano y no bloquea la terminal. Esto permite continuar utilizando la terminal mientras el proceso está en ejecución.

Caso C - python 2_puntos.py & disown

C.1 Las herramientas de la terminal	
1. `python 2_puntos.py & disown`. ¿Qué imprime? Anota el PID: 3650	Imprime el PID y comienza a ejecutar en segundo plano. Los puntos se dibujan de una forma más rápida.
2. `jobs`. ¿Lo ve?	No se muestra en la lista de trabajos.
3. `kill %1`. ¿Qué responde?	-bash: kill: %1: no such job
4. `bg`. ¿Qué responde?	-bash: bg: current: no such job
5. `fg`. ¿Qué responde?	-bash: fg: current: no such job
C.2 ¿Sobrevive a salir de la terminal?	
6. Cierra la ventana de la terminal y vuelve a entrar	Se cerro la sesión con `CTRL+D` y se volvio a ingresar vía ssh.
7. `ps aux | grep py` (1). ¿Sobrevivió? ¿Es el mismo PID del paso 1?	Sí, es el mismo PID. Muestra: orangepi 3650 87.5 3.0 232488 30196 ? Rl 03:00 3:27 python 2_puntos.py lo cual indica que el proceso sigue corriendo.
C.3 Las herramientas del sistema	
8. `kill -STOP <PID>`. ¿Los puntos?	Se detuvieron.
9. `kill -CONT <PID>`. ¿Los puntos? ¿Volvió al primer plano?	Los puntos se reanudaron y volvieron al primer plano.

10. `kill <PID>`, ¿Los puntos?	Los puntos volvieron a detenerse.
11. `ps aux | grep py` (1). ¿Qué devuelve ahora?	Ya no aparece en la lista de procesos, ya que el proceso fue terminado.

¿Qué herramientas dejaron de funcionar, y cuáles siguieron sirviendo?

Las herramientas que dejaron de funcionar fueron `jobs`, `fg` y `bg`. Las herramientas que siguieron funcionando fueron `ps` y `kill`. Esto se debe a que al usar `disown`, el proceso se desvincula de la terminal, por lo que ya no es considerado un trabajo de la terminal actual.

Caso D - python -u 3_calibracion.py > salida.log 2>&1 & disown

D.1 Sin redirigir la salida	
1. `python 3_calibracion.py & disown`. Calibra las cuatro esquinas. ¿Qué imprime en cada toque?	imprime las coordenadas correspondientes a cada toque de la pantalla
2. Cierra la ventana de la terminal (directo sin CTRL+C) y vuelve a entrar	Se cerró utilizando el atajo `CTRL+D`.
3. `ps aux | grep py` (1). ¿Sobrevivió al cierre?	Sí, sobrevivió al cierre y conservó el PID 4225.
4. Toca la pantalla. ¿Dibuja el punto donde tocaste?	No, no dibuja nada ni tampoco imprime las coordenadas.
5. `ps aux | grep py` (1). ¿Y ahora? Si quedó vivo, detenlo con `pkill -f 3_calibracion.py`	No se registra ningún toque; se eliminó el proceso correctamente.
D.2 Mandando la salida a un archivo	
6. Lánzalo con la línea completa. ¿Imprime algo en la terminal?	Solo imprime el PID del proceso.
7. Calibra las cuatro esquinas y dibuja en la pantalla. ¿Escribe algo en la terminal?	No escribe nada en la terminal ya que la salida se redirige a un archivo.
8. `cat salida.log`. ¿Qué hay adentro?	Se encuentran las coordenadas de los toques realizados durante la calibración.
9. Cierra la ventana de la terminal y dibuja en la pantalla. ¿Funciona?	Si esta funcionando correctamente.
10. Vuelve a entrar	Se ingreso correctamente.

11. `ps aux | grep py` (1). ¿Sigue corriendo?	Si sigue corriendo. con el PID de 4225
12. Detenlo con `pkill -f 3_calibracion.py`	Se detuvo obteniendo:

Los dos bloques hicieron exactamente lo mismo. ¿Por qué acabaron distinto?

En el primer bloque, la salida del programa se dirigió a la terminal, por lo que al cerrar la terminal, el proceso perdió su salida estándar y no pudo continuar funcionando correctamente. En el segundo bloque, la salida se redirigió a un archivo (`salida.log`), lo que permitió que el proceso continuara ejecutándose incluso después de cerrar la terminal, ya que no depende de la terminal para su salida.

Caso E - screen

E.1 Dejarlo trabajando	
1. `screen -S practica`. ¿Qué cambió en la terminal?	Se creó una nueva sesión, dezplasando la terminal.
2. Adentro: `cd ~/Documents/1_Pantalla` y `source ~/Documents/venv_embebidos/bin/activate`	Se activó el entorno virtual de python.
3. `python 3_calibracion.py`. Calibra y dibuja en la pantalla. ¿Imprime en la terminal?	Se imprime en la terminal, el punto corresponde a la posición donde se tocó la pantalla.
4. `Ctrl+A` y luego `D`. ¿Qué dice? ¿Dónde quedaste?	[detached from 4649.practica] además de que indico que no existia otra ventana. Me encuentro en la sesión original.
5. `screen -ls`. ¿Qué lista?	Indica las sesiones activas
E.2 Después de cerrar la terminal	
6. Cierra la ventana de la terminal (directo sin Ctrl+C) y dibuja en la pantalla. ¿Funciona?	Funciona, sin ninguna interrupción
7. Vuelve a entrar	Al ingresar sigue funcionando sin cambio aparente.

8. `ps aux | grep py` (1). ¿Sigue corriendo?	Se encuentra ejecutandose en mi plano desde su terminal, tambien indica que esta suspendido esperando una entrada e indica que esta empleando muúltiples hilos
9. `jobs`. ¿Lo ve?	No lo ve.
10. `screen -ls`. ¿Sigue ahi la sesión?	La sesión sigue activa.
11. `screen -r practica`. ¿Aparece lo que dibujaste con la terminal cerrada?	Al reingresar aparece lo que se dibujó.
12. Toca la pantalla. ¿Lo imprime en vivo?	Se actualiza en tiempo real.
13. `Ctrl+Z` y luego `jobs`. ¿Lo ve ahora?	Al ingresar `CTRL+Z` el proceso se suspende y en la lista de jobs aparece como [1] Stopped
14. `fg` y luego `Ctrl+C`. ¿Puedes detenerlo desde aquí?	Si, al usar `fg` se vuelve a traer el proceso al frente y con `Ctrl+C` se puede detener.
15. `exit` para cerrar la sesión de screen	al cerrar la sessión e ingresar nuevamente screen -ls, ya no aparece en la lista. Por lo que la otra sesión se cierra correctamente.

Compara los pasos 9 y 13: el mismo comando, dos respuestas. ¿Dónde estabas en cada uno?

La diferencia entre los pasos 9 y 13 es que en el paso 9, el comando `jobs` se ejecutó en la terminal original, donde no había ningún proceso en segundo plano asociado a esa terminal, por lo que no se mostró ningún trabajo. En el paso 13, el comando `jobs` se ejecutó dentro de la sesión de `screen`, donde el proceso estaba suspendido, por lo que sí se mostró como un trabajo detenido.

Sintesis

Letra	Caso	Comando	Opción	Aplicación principal
A	Caso A	`python 2_puntos.py`	(C)	Dejar el programa trabajando y poder desconectarse de la terminal
B	Caso B	`python 2_puntos.py &`	(A)	Correr el programa en la sesión actual y nada más: probarlo y detenerlo
C	Caso C	`python 2_puntos.py & disown`	(E)	Volver a ver la salida en vivo y recuperar el control del programa
D	Caso D	`python -u 3_calibracion.py > salida.log 2>&1 & disown`	(B)	Seguir usando la terminal mientras el programa corre
E	Caso E	`screen`	(D)	Darle a un programa que imprime un lugar donde vaciar su salida sin fallar

¿Qué casos conviene usar si el programa imprime en la terminal? Encierra los que apliquen.

Los casos que más convienen usar si el programa imprime en la terminal son los casos A, B y E. Esto se debe a que estos casos permiten ver la salida del programa en tiempo real y también permiten interactuar con él mientras se ejecuta. En cambio, el caso C al utilizar `disown` desvincula el proceso de la terminal, por lo que no se puede ver la salida en tiempo real. El caso D redirige la salida a un archivo, por lo que no se puede ver la salida en la terminal.

Conclusiones

El uso de herramientas como `jobs`, `fg`, `bg`, `kill` y `screen` permite un control más eficiente de los procesos en ejecución en la terminal. La práctica demostró cómo diferentes métodos de ejecución y manejo de procesos afectan la interacción con la terminal y la persistencia de los procesos al cerrar la sesión.

El caso de redirección de salida a un archivo mostró la importancia de gestionar correctamente la salida estándar y de error para mantener la funcionalidad del programa incluso después de cerrar la terminal. Por otro lado, el uso de `screen` proporcionó una solución robusta para mantener

procesos activos y accesibles, permitiendo desconectarse y reconectarse sin perder el estado del programa.

Como conclusión general, la práctica enfatizó la necesidad de comprender y aplicar correctamente las herramientas de gestión de procesos para optimizar el flujo de trabajo en entornos de desarrollo y administración de sistemas.